

Operator ETL

Local-First Data Intake, Warehouse, and Insights

Proposal & System Specification — v1

Document	Operator ETL Proposal
Version	1.0
Date	August 17, 2026
Status	Proposed / Ready for review
Repository	/Users/Sour/operator-etl

Executive Summary

Operator ETL is a local-first data intake system designed to move raw files and API payloads through a trustworthy pipeline into actionable metrics. It follows the medallion architecture (bronze → silver → gold), validates every row before it enters analytics tables, and withholds insights when data quality fails.

The system is built as reusable infrastructure, not a one-off script. New data sources plug in through a registry; the pipeline runner, quality gates, and insight layer stay unchanged.

One-line pitch: Drop a CSV or call an API → store raw data immutably → type and clean valid rows → compute SQL metrics → surface KPIs in CLI or dashboard — with bad rows quarantined and insights blocked when quality thresholds are not met.

Problem Statement

Most ad-hoc data workflows fail in predictable ways:

1. Silent bad data — invalid rows load into dashboards without visibility.
2. No idempotency — re-running the same file double-counts records.
3. Tight coupling — each new source requires a new script or app.
4. Overconfident outputs — charts render even when upstream data is broken.
5. No audit trail — operators cannot answer *what ran, when, and what failed*.

Operator ETL addresses these with explicit layers, quarantine paths, run logging, and fail-closed quality gates.

Goals

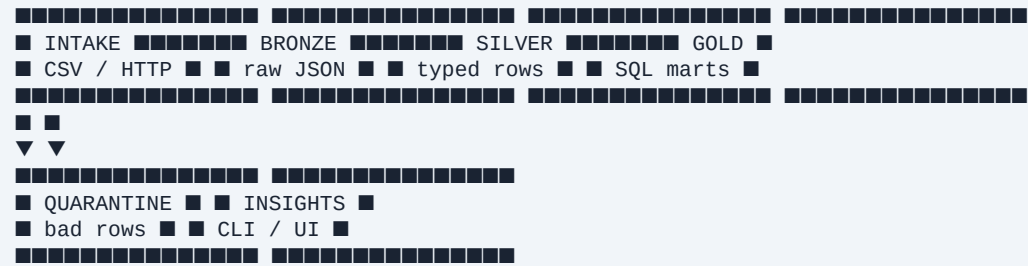
Goal	Description
Trustworthy intake	Raw payloads preserved; validation before silver
Operator clarity	Run logs, quarantine reasons, quality metrics
Extensibility	New sources via registry, not rewrites
Local-first	No cloud dependency for v1
Deterministic core	SQL and Python own ETL; no LLM in the critical path

Non-goals (v1)

- Cloud warehouse or hosted dashboard
- Airflow, Spark, Kafka, or streaming
- LLM-generated insights
- Multi-tenant SaaS
- Domain-specific adapters (Substack, finance, etc.) — supported later via registry

System Architecture

High-level flow



Layer responsibilities

Layer	Responsibility	Technology (v1)
Extract	Read files/APIs, compute content hash	Python, httpx
Bronze	Store immutable raw rows + metadata	DuckDB
Transform	Validate, type, deduplicate	Pydantic v2
Quarantine	Isolate invalid rows with error reasons	DuckDB

Layer	Responsibility	Technology (v1)
Gold	Aggregated metrics via SQL	DuckDB marts
Insights	Present KPIs when quality passes	CLI, Streamlit
Orchestration	Run pipeline, log outcomes	Typet CLI

Design principle

Agents orchestrate in future versions; ETL stays deterministic in v1.

The data plane (extract, load, validate, aggregate) is pure Python and SQL. An optional agentic layer can be added later for schema mapping, narrative insights, and human-in-the-loop review — without replacing the warehouse or quality gates.

Data Sources

Sources are declared in a YAML registry ([pipelines/demo.yaml](#)). Each entry specifies kind, path or URL, and target tables.

Source name	Kind	Input	Purpose
demo	csv	Bundled sample file	Demo and testing
inbox	csv_dir	Drop folder (drops/inbox/)	Operator CSV intake
http	http	GET JSON list or nested object	API intake stub

Adding a new source requires one registry entry and, if needed, a schema contract. The pipeline runner does not change.

Extract behavior

- CSV: Parse rows as string dictionaries; UTF-8 with BOM support.
- CSV directory: Scan for [.csv](#) files; extract each independently.
- HTTP: GET JSON; accept top-level list or { [data](#) | [items](#) | [orders](#) | [results](#): [...] }.
- Offline stub: [file](#): URLs read local JSON for development without network.

Every extract produces: [file_name](#), [content_hash](#) (SHA-256), and [rows\[\]](#).

Warehouse Schema

Operational tables

Table	Purpose
<code>ingest_files</code>	One row per unique file hash (idempotency)
<code>pipeline_runs</code>	Audit log: run_id, source, timestamps, row counts, status

Medallion tables

Table	Contents
<code>bronze_raw</code>	Raw JSON payload + <code>_content_hash</code> , <code>_source</code> , <code>_ingested_at</code> , <code>_row_num</code>
<code>silver_orders</code>	Validated, typed business rows
<code>quarantine_orders</code>	Rejected rows + error message

Gold marts

Mart	Metrics
<code>gold_kpis</code>	Order count, customer count, revenue, avg order, latest order, freshness
<code>gold_volume_daily</code>	Orders and revenue by day
<code>gold_top_skus</code>	Top SKUs by revenue
<code>gold_quality</code>	Bronze/silver/quarantine counts, quarantine rate, last ingest

Silver Contract (Validation)

Each bronze row is validated against a typed contract before entering silver.

Field	Type	Rules
<code>order_id</code>	string	Required, non-empty
<code>customer_id</code>	string	Required, non-empty
<code>ordered_at</code>	datetime	Parseable ISO timestamp
<code>amount</code>	float	Must be greater than zero
<code>sku</code>	string	Required
<code>status</code>	string	Required

Outcomes per row:

- Valid → insert into [silver_orders](#)
- Invalid → insert into [quarantine_orders](#) with reason
- Duplicate [order_id](#) → quarantine (no silent overwrite)

The pipeline continues on bad rows; it does not crash the entire run.

Idempotency & Reliability

Guarantee	Mechanism
No duplicate file loads	SHA-256 hash recorded in ingest_files ; re-ingest skips
No duplicate business keys	order_id uniqueness enforced → quarantine on conflict
Bad rows isolated	Quarantine table + quality gate
Traceable runs	pipeline_runs with row counts and status
Extensible sources	YAML registry + pluggable extractors

Quality Gate (Fail-Closed Insights)

Before KPIs are shown, the system evaluates data quality. Insights are withheld when gates fail.

Check	Default threshold	On failure
Quarantine rate	≤ 35% of bronze rows	KPIs blocked
Freshness	Last ingest ≤ 7 days	KPIs blocked
Silver row count	> 0	KPIs blocked

Configurable via environment:

- [OPERATOR_ETL_MAX_QUARANTINE_RATE](#)
- [OPERATOR_ETL_MAX_FRESHNESS_HOURS](#)

When blocked, operators still see quality metrics and quarantine details — only headline KPIs and charts are hidden.

Operator Interface

Commands

Command	Action
<code>etl ingest --source <name></code>	Bronze load only
<code>etl run --source <name></code>	Full pipeline: ingest → silver → gold → insights
<code>etl insight</code>	Print KPIs (respects quality gate)
<code>etl sources</code>	List registered sources
<code>etl dashboard</code>	Open Streamlit run inspector

Output surfaces

1. CLI — Text KPIs, daily volume, top SKUs, quality summary.
2. Streamlit dashboard — Quality panel, KPI cards, time series, SKU breakdown, pipeline run history.

Both read from the same gold tables. One source of truth.

Demo Scenario

A sample CSV contains 21 order rows:

Outcome	Count	Examples
Silver (valid)	17	Normal orders
Quarantined	4	Empty order_id, invalid date, negative amount, non-numeric amount

After `etl run --source demo`:

- Quarantine rate: 19.1% (under 35% threshold) → quality gate PASS
- KPIs render: revenue, order count, daily volume, top SKUs
- Re-running the same file: 0 new rows (hash skip)

This demonstrates idempotency, quarantine visibility, and fail-closed quality in one walkthrough.

Technology Stack

Component	Choice	Rationale
Language	Python 3.12	Data ecosystem, Pydantic, DuckDB bindings
Warehouse	DuckDB	Local SQL, zero infra; portable to Postgres later

Component	Choice	Rationale
Validation	Pydantic v2	Explicit contracts, clear error messages
HTTP client	httpx	Modern async-capable client for API extract
CLI	Typer	Typed commands, good UX
Dashboard	Streamlit	Fast local visualization
Packaging	uv + pyproject.toml	Reproducible environments
Tests	pytest	Pipeline, idempotency, HTTP, quality gate

Repository Layout

```

operator-etl/
├── README.md
├── pyproject.toml
├── pipelines/demo.yaml # Source registry
├── samples/ # Demo CSV + HTTP JSON
├── drops/inbox/ # Operator drop folder
├── sql/marts/ # Gold SQL definitions
├── dashboard/app.py # Streamlit UI
├── warehouse/ # operator.duckdb (gitignored)
├── src/operator_etl/
│   ├── cli.py
│   ├── config.py
│   ├── pipeline.py
│   ├── sources.py
│   ├── extract/ # csv, http
│   ├── load/ # duckdb
│   ├── transform/ # contracts, clean
│   ├── insights/ # metrics, quality gate
│   └── tests/

```

Success Criteria

v1 is complete when:

1. A CSV dropped in [drops/inbox/](#) runs end-to-end with one command.
2. Invalid rows land in quarantine, not silver.
3. KPIs show freshness, volume, and SKU breakdown when quality passes.
4. Re-ingesting the same file produces zero duplicate rows.
5. Quality gate blocks KPIs when thresholds fail.
6. HTTP source loads JSON list into the same pipeline.

All criteria are met in the current implementation.

Phase	Capability
v1.1	Additional source adapters (Substack export, finance CSV)
v1.2	DuckDB → Postgres/Supabase; scheduled runs (Dagster/Prefect)
v2.0	Agentic layer: LangGraph orchestration, PII scan/tokenize, schema-map agent, insight narrative with critic/faithfulness check, human-in-the-loop interrupts
v2.1	Hosted dashboard on Vercel; observability (Langfuse/LangSmith)

Evolution: Agentic Layer (Conceptual)

```

■ CONTROL PLANE (v2) ■
■ LangGraph · checkpoints · HITL · tool allowlists · critic ■
■
■
■ POLICY PLANE (v2) ■
■ PII scan · token vault · redacted views · spend budgets ■
■
■
■ DATA PLANE (v1 – unchanged) ■
■ Extract · bronze · silver · quarantine · gold SQL ■

```

- LLMs never run unconstrained SQL on raw rows.
- PII values never enter graph state or traces — metadata only.
- Critic node verifies every narrative number against gold metrics.
- Checkpoints enable resume after failure or human approval.

Recommendation

Proceed with Operator ETL v1 as the foundation for local data intake and operator insights. The architecture is intentionally boring in the data plane and extensible at the edges (sources, marts, future agent layer).

Suggested next steps for review:

- 1. Confirm canonical schema covers first real data source (orders vs. events vs. subscribers).
- 2. Approve quality gate thresholds for production use.
- 3. Prioritize v1.1 source adapter (inbox-only vs. specific API).
- 4. Decide whether v2 agentic layer is in scope for Q4 or deferred.

Appendix: Test Coverage Summary

Test area	Validates
Idempotent ingest	Same file hash skipped on re-run
Quarantine	Invalid rows isolated with reasons
Gold marts	KPIs, volume, SKU breakdown built
Quality gate	KPIs blocked when quarantine rate exceeds threshold
HTTP source	JSON list lands in warehouse
Source registry	demo, inbox, http registered

Current status: 9 tests passing.

End of document